



Assignment Cover Sheet

Learner Name: Samuel Simeon
Learner Number: 3090681
Faculty: Computing
Programme: BSCH Stage/Year: 4
Module: Cloud Services & Platforms
Study Mode: Full Time ☒ Part-time ☐
Lecturer Name: Sahrish Zahoor
Assignment Title: Building an Implementation of Dropbox
Additional Relevant Information (e.g. number of pieces submitted etc.):

AI in Learner Assessment Policy Indicate here applicable categories allowed for this assignment.	Category of AI Use	Allowed or not allowed
	No AI Use	
	AI for Planning	Allowed
	AI for Editing	(NOT ALLOWED)
	AI for Support Tasks	(NOT ALLOWED)
	AI for Collaboration	(NOT ALLOWED)
	Full AI Use	(NOT ALLOWED)

Academic Integrity Honour Code:

I submit this work in line with the principles of Academic Integrity as appearing in the [Academic Integrity policy](#), the assessment description(s), and the relevant [AI Assessment Scale](#) in the AI in Learner Assessment

I affirm that I have not given or received any *unauthorised* help, from a person or through unauthorised content generation on this assignment, and that this work is my own.

I understand that penalties may be imposed if this assessment is in breach of the [Academic Integrity and Misconduct policy](#).

Signature: Samuel Simeon Date: 25/04/2026

Please note: Learners **MUST** retain a hard/soft copy of all assignments and must have the lecturer/member of Faculty acknowledge/sign as proof of submission.

Cloud Dropbox Assignment

1. General Approach

My approach was to build the application as a simplified cloud storage service with a clear separation between authentication, metadata storage, file storage, backend API logic, and frontend interaction. I used Fast API as the main backend framework because it makes it easy to organise routes and connect them to the frontend through JSON-based requests and responses.

The project was designed around four main services. Firebase Authentication handles user signup, login, and identity verification. MongoDB Atlas stores metadata such as folder structures, file details, sharing records, and deletion state. Azurite acts as the local file storage service, storing the actual uploaded file contents as blobs. The frontend, built with HTML, CSS, and JavaScript, communicates with the backend using fetch requests.

I implemented the system incrementally, authentication and database first, then folder creation and directory navigation, then file upload and download, and finally file sharing, trash and restore, and duplicate detection.

A key design decision was to separate file content from file metadata. File contents are stored in Azurite, while information about the files, filename, folder location, size, hash value, and sharing state is stored in MongoDB Atlas. Another important decision was using SHA-256 hashing to detect duplicates, which is more reliable than checking only filenames because two files can have different names but identical content.

2. System Architecture

The system follows a client-server structure. The frontend runs in the browser and communicates with the Fast API backend through HTTP requests. The backend processes each request, verifies the user, interacts with the database and blob storage, and returns the result to the frontend.

The architecture has four main layers. The first is the frontend presentation layer HTML templates, CSS styling, and JavaScript logic which displays data and captures user actions. The second is the FastAPI application layer, which defines all routes and contains the main logic for handling requests.

The third is the metadata layer using MongoDB Atlas. MongoDB stores structured records for folders, files, shares, and deleted items. Folder records include the folder name, owner, and parent folder relationship. File records include filename, owner, folder location, hash value, blob name, and deletion state. Share records connect a file to another user.

The fourth is the storage layer using Azurite, which stores actual file contents as blobs. The backend creates a unique blob name for each upload. When a user downloads a file, the backend reads the correct blob and streams it back to the client.

Authentication is handled separately through Firebase. The frontend obtains a Firebase ID token and sends it in protected requests. The backend validates the token before allowing

access to any route, ensuring that folder and file operations are always linked to the correct user.

3. Backend Modules and How They Work Together

The backend is organised into focused modules so that each file handles a specific responsibility.

main.py

This is the entry point of the FastAPI application. It creates the FastAPI app, mounts the static files, loads the HTML templates, and includes the route modules. It also starts a background task on startup that runs hourly to permanently delete items that have been in the trash for over one hour.

auth.py

This module is responsible for user authentication on the backend. Its main purpose is to validate Firebase ID tokens sent from the frontend. The key function verifies the token, extracts the authenticated user details, and makes that information available to protected routes. All other modules depend on this so that folder, file, and sharing operations are always linked to the correct user.

db.py

This module manages the MongoDB Atlas connection. It creates the database client and provides access to the database used by the rest of the application. Route files import the database object from here rather than creating their own connections, which keeps things consistent.

storage.py

This module manages the Azurite blob storage connection. It creates the container client used to upload, download, and delete file blobs. Since the database only stores metadata, this module is what allows the application to work with actual file data.

models/

The models folder stores Pydantic request models. These define the expected structure of input data, such as folder creation requests and sharing requests, and validate incoming request bodies so the route code stays clean.

routes/

This folder contains the main API logic. `folders.py` handles creating, listing, and deleting folders. `files.py` handles uploading, downloading, deleting, restoring, and permanently deleting files. `directory.py` returns the current folder contents, the trash view, and duplicate groups. `shares.py` handles sharing files with other users and giving recipients access to download them.

services/trash.py

The services folder contains supporting logic that does not belong directly in the routes. The trash service manages soft deletion and restoration of files and folders. This keeps the route files simpler by moving repeated deletion logic into a dedicated place. It also contains the purge function that permanently deletes items older than one hour from the trash.

4. Frontend Modules and How They Work Together

The frontend is built with HTML, CSS, and JavaScript. The login and signup pages use the Firebase Web SDK to create accounts or sign in, then redirect to the main dashboard. The dashboard layout includes the sidebar navigation, the file area, and the preview panel, giving the application a cloud-storage style interface.

The CSS file controls the visual design sidebar layout, button styles, tables, cards, preview panels, and duplicate highlighting and was designed to make the application feel more like a real storage system.

The main dashboard script is the largest frontend module. It loads folder contents, handles navigation through the directory tree, uploads and downloads files, triggers share requests, opens the trash view, restores deleted files, and loads duplicate groups. It also tracks the currently selected file and the current folder so the interface always reflects where the user is.

The frontend and backend communicate through fetch requests. When the user performs an action, the JavaScript sends a request to the corresponding backend route with the Firebase token attached. The response is then used to update the frontend, for example by reloading the directory or showing a toast message.

5. Key Features and Logic

The folder system allows users to create top-level folders and nested subfolders. Each folder stores a parent reference so the backend can reconstruct the directory tree. A folder can only be deleted if it is empty the backend checks for child files and subfolders before allowing deletion and blocks the request if either exists.

The file management system handles uploads, downloads, deletion, restoration, and permanent removal. When a file is uploaded, the content goes to Azurite and the metadata goes to MongoDB. Deleted files are moved to Trash first, allowing recovery before permanent removal.

The sharing system allows a user to share a file with another registered user by email. The backend looks up the recipient through Firebase Authentication and stores a sharing record in MongoDB. The recipient can view and download the shared file from a separate section without being the file owner.

Duplicate detection is one of the more advanced features. Each uploaded file is hashed using SHA-256. If a file with the same hash already exists in the same folder, the upload is blocked. The system can also scan the user's entire Dropbox and group together files that share the same hash, displaying the matching paths so the user can see where duplicates exist.

6. Challenges Faced and How I Solved Them

One of the first challenges was getting Firebase Authentication working correctly on the backend. The frontend was generating tokens fine, but I had to make sure the backend was verifying them on every protected route. I solved this by creating a dedicated `auth.py` module with a single `get_current_user` function that all routes depend on, so the verification logic only had to be written and tested once.

Another challenge was implementing the trash and restore system. The tricky part was making sure that when a file is deleted and then restored, it goes back to the correct folder. I solved this by storing the original folder ID on the file document at the point of deletion, so the restore function always knows where to put it back.

7. Summary

This project was developed as a simplified cloud storage application that combines user authentication, file storage, metadata management, sharing, trash recovery, and duplicate detection into a single system. The design separates responsibilities across Firebase Authentication, MongoDB Atlas, Azurite, FastAPI, and the frontend interface, which makes the application easier to understand and maintain.

Features such as blocked duplicate uploads, global duplicate detection, file restoration, and folder deletion checks helped move the project beyond simple upload and download behaviour. Overall, the final system demonstrates how multiple cloud technologies can work together and shows the importance of modular design and clear interaction between frontend and backend components.